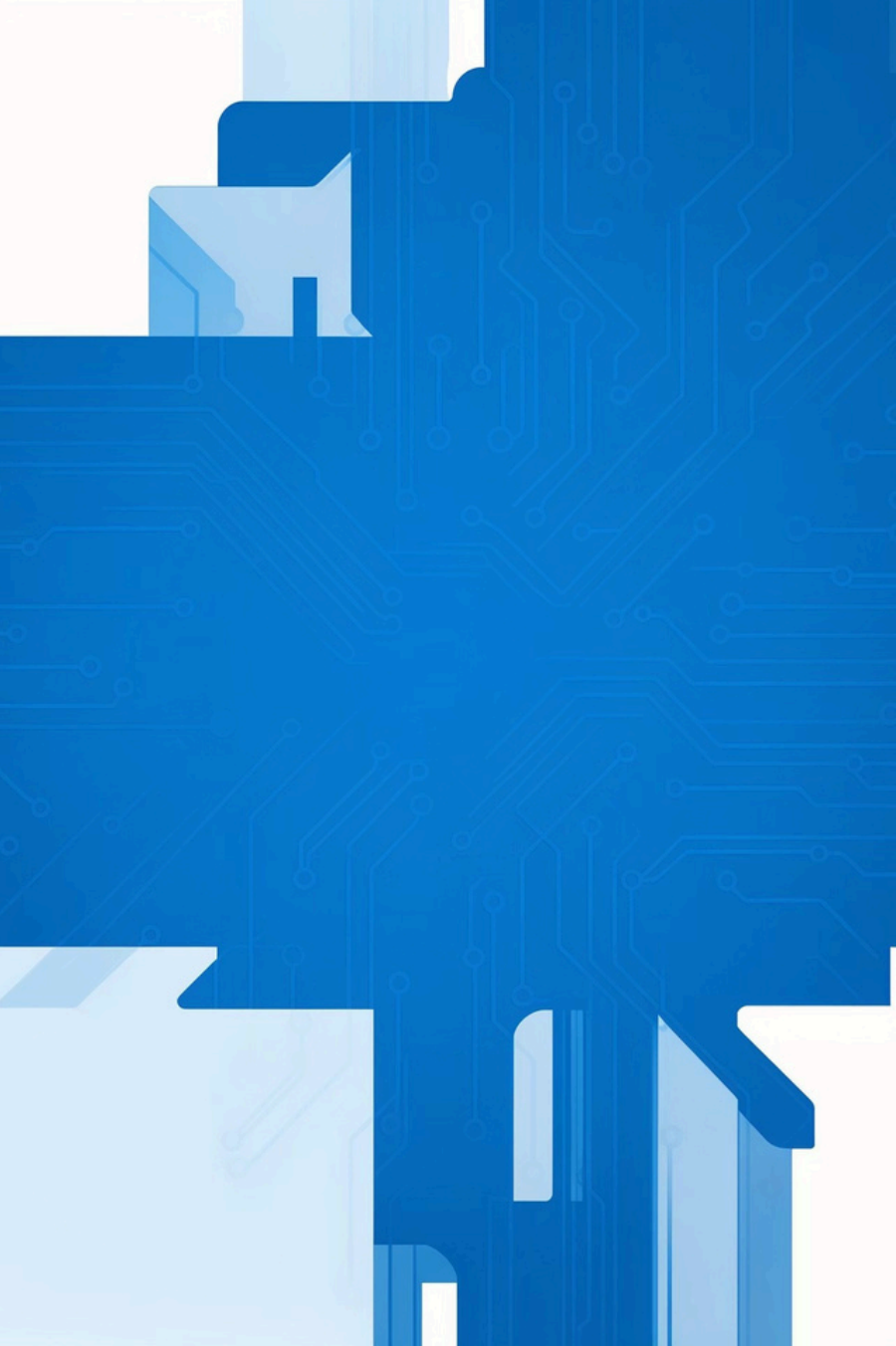




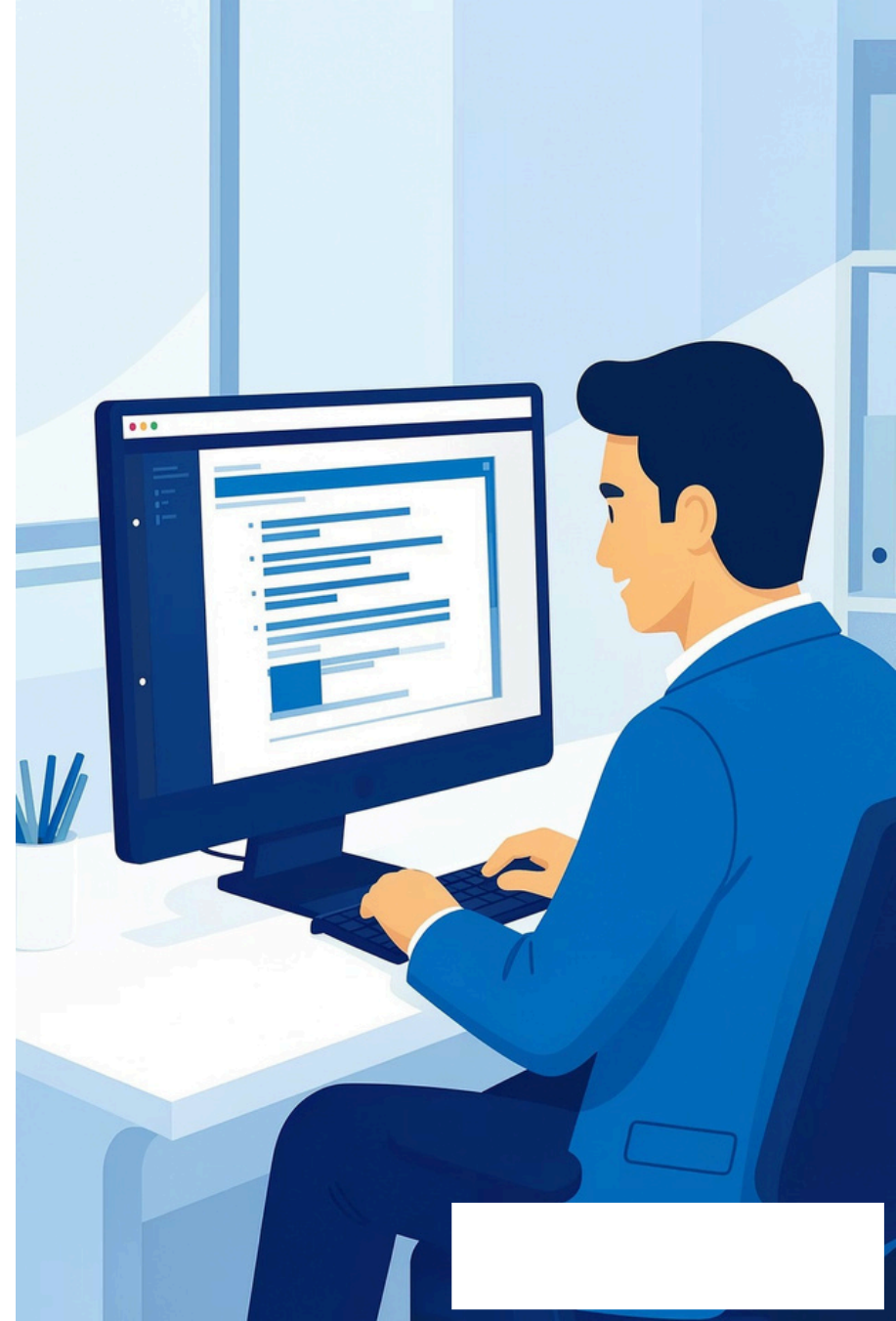
V-Model (Verification & Validation Model)

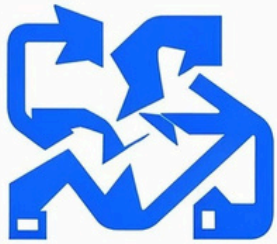
Team Members :

Manu Pratap Singh,
Manish Yadav,
Ruchika Sharma,
Anuj Yadav

Introduction

- Definition: A sequential development model emphasizing paired verification and validation activities.
- Designed to overcome key limitations of the Waterfall model.
- Emphasises early and continuous testing aligned with development phases.
- Improves overall software quality and traceability.





Late Stage
Workflow
TESTING DELAYS



HOEKSCETA TO RENDER.
IOUT TB COMVN. CUVTE&L U:3Y



Why the V-Model?

- Problems with Waterfall: late discovery of defects and prolonged feedback loops.
- Late testing: testing mostly after implementation.
- High defect fixing cost when issues are found late.
- Necessitates early testing and verification to reduce risk.

Structure of the V-Model

- V-shaped lifecycle: development on left, coding at bottom, testing on right.
- Left side (Verification): planning and design activities.
- Bottom (Coding) : implementation.
- Right side (Validation): unit → integration → system → acceptance testing.



Verification Phase (Left side)

- Requirement Analysis: capture functional & non-functional needs.
- System Design: define system architecture and interfaces.
- High-Level Design (HLD): module boundaries and data flow.
- Low-Level Design (LLD): detailed algorithms and data structures.
- Goal: Build the product *correctly*.



Coding & Validation Phase (Right side)

- Coding: translate LLD into executable code.
- Unit Testing: verify individual components (LLD ↔ Unit).
- Integration Testing: verify interfaces and combined modules (HLD Integration).
- System & Acceptance Testing: validate full system against requirements.



Verification



Your lot that a vailed the effect failure, con cert. the sloncul if verifie cow pallettic frat you rahing, correct level oh, and nandent.

Hard constraint: no object figure from stylis form maties tue on reference images.

Validation



Wey an traict, no obects figures compangerts from or reference competitions from in the im the no extra text.

Hard constraint: no objects only the ner what that the text text creasest in decrupt

Verification vs Validation

- Verification: "Are we building the product correctly?" — checks artifacts, designs.
- Validation: "Are we building the right product?" — checks behaviour against requirements.
- Phase mappings:
- Requirement Analysis ↔ Acceptance Testing
- System Design ↔ System Testing
- High-Level Design ↔ Integration Testing
- Low-Level Design ↔ Unit Testing

Advantages

- Earlydefectdetection: verification begins with requirements.
- Improved software quality through parallel test planning.
- Reduced development cost by catching issues early.
- Better planning and clearer deliverables per phase.
- Easy project tracking and high reliability for critical systems.



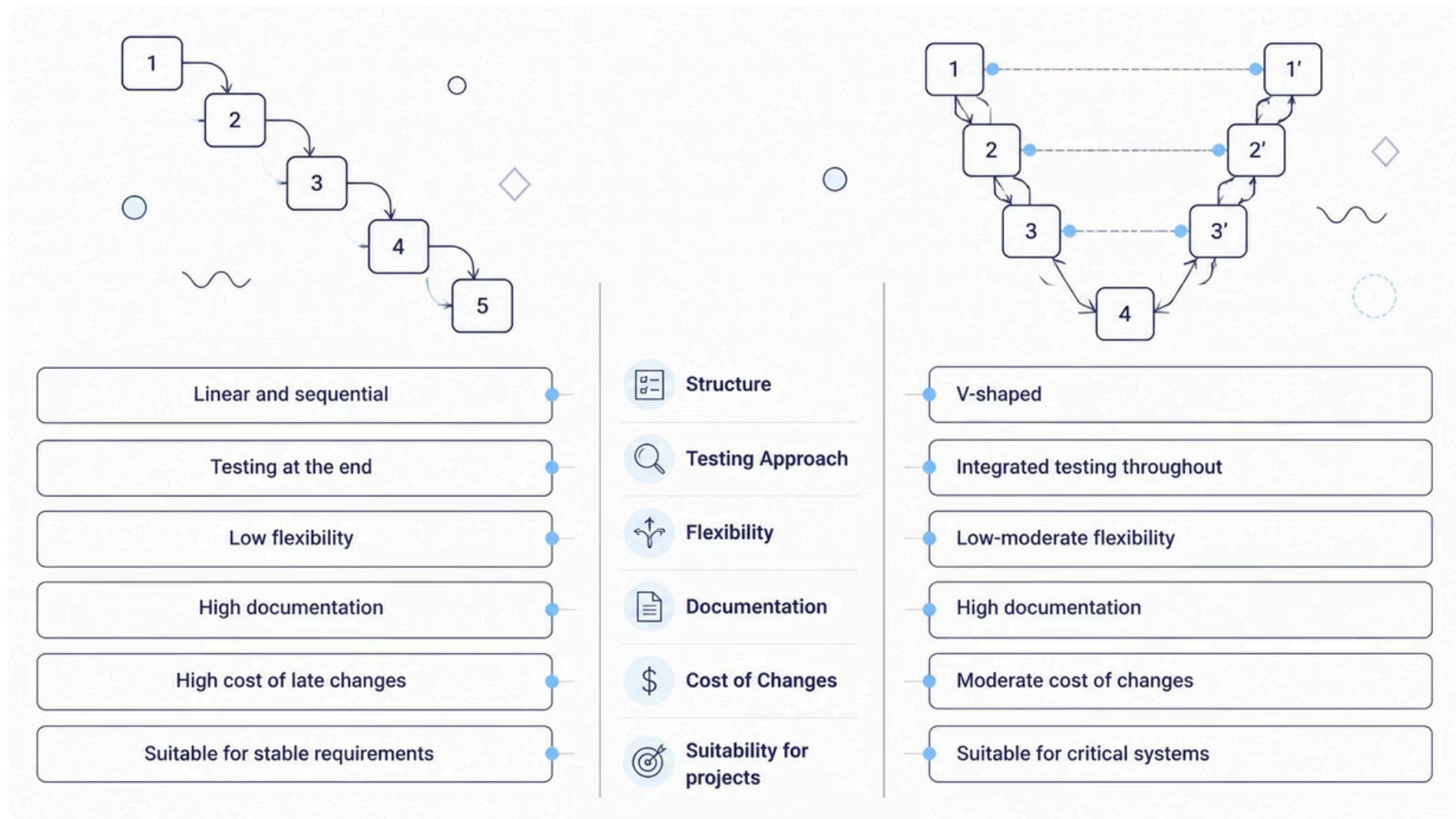


Limitations

- Not suitable for changing requirements: inflexible to change.
- Rigid process: strict phase-to-phase flow.
- High documentation overhead for each verification level.
- Expensive for small projects and no prototyping by default.

Waterfall vs. V-Model: A Comparative Overview

Understanding the key differences between the Waterfall and V-Model helps in selecting the right approach for software development based on project needs and risk tolerance.



While both models are sequential, the V-Model's integrated testing approach significantly improves defect detection and reduces the impact of changes compared to the traditional Waterfall approach.

Conclusion: Key Takeaways

Integrated Quality Assurance

The V-Model seamlessly integrates testing with development from initial requirements, moving beyond Waterfall's late-stage testing.

Early Defect Detection

Proactive verification and validation phases ensure issues are discovered and resolved significantly earlier, reducing overall project costs.

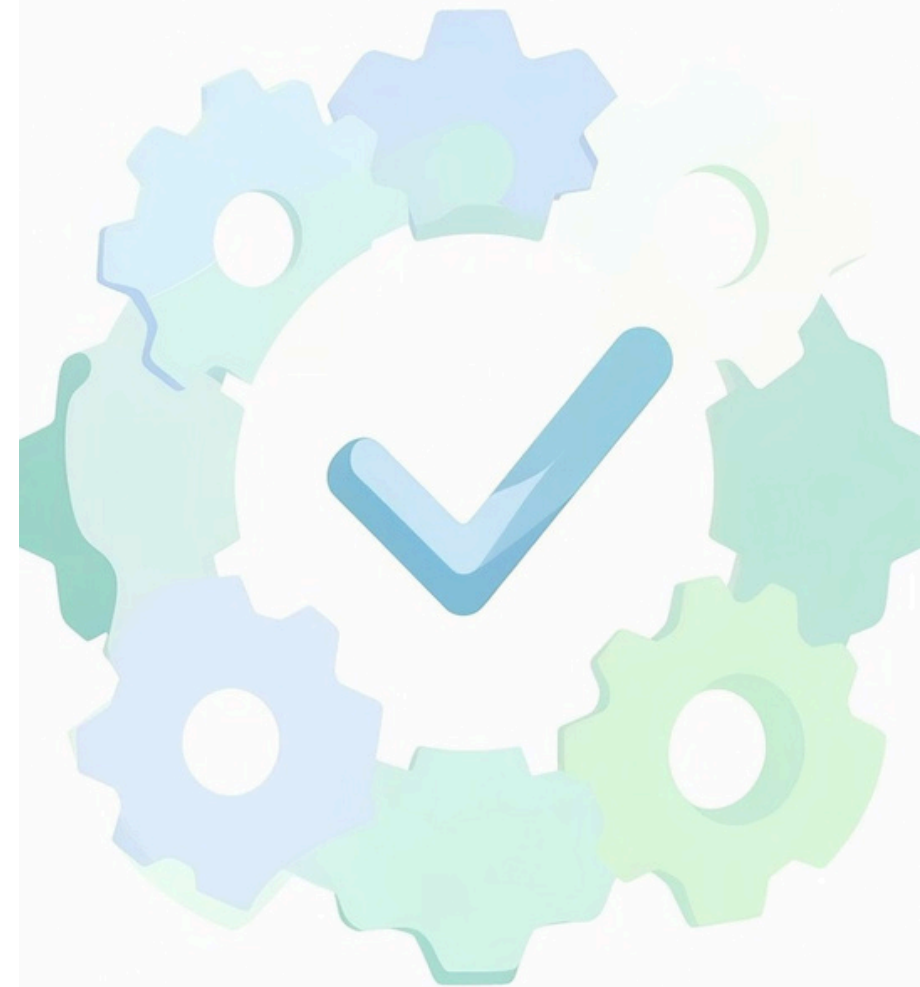
Enhanced Traceability

Its V-shaped structure provides clear, direct mappings between each development activity and its corresponding testing phase, ensuring comprehensive coverage.

Ideal for Critical Systems

With its rigorous, structured approach, the V-Model is particularly well-suited for projects requiring high reliability and stable, well-defined requirements.

Choosing the V-Model strategically empowers teams to deliver high-quality, reliable software, particularly in environments where precision and early validation are paramount.



Thank You

c